SOFTWARE TESTING REPORT

TUTORIAL – 11

Python Automation Testing with Selenium WebDriver

SauceDemo Web Application | Browser Navigation, Product Selection, Cart and Checkout Workflow

Student Name	Harshita Malviya
Student ID	2504030100426
Practical	Tutorial 11 — Python Automation Testing
Application Under Test	SauceDemo (saucedemo.com)
Tools	Python, Selenium WebDriver, Google Chrome, PyCharm

Prepared as a practical execution report using the supplied Tutorial 11 manual and screenshots.

1. Aim

To analyze and demonstrate Python-based automation testing for website navigation, login form handling, product selection, cart navigation, checkout form handling, multi-page workflow validation, and browser back/forward navigation.

2. Test Environment

Parameter	Details
Application	SauceDemo Web Application
Automation language	Python
Automation framework/tool	Selenium WebDriver
Browser	Google Chrome
IDE / execution environment	PyCharm (virtual environment shown in console screenshot)
Wait strategy	WebDriverWait with a 100-second timeout in the supplied script

3. Test Scenario and Procedure

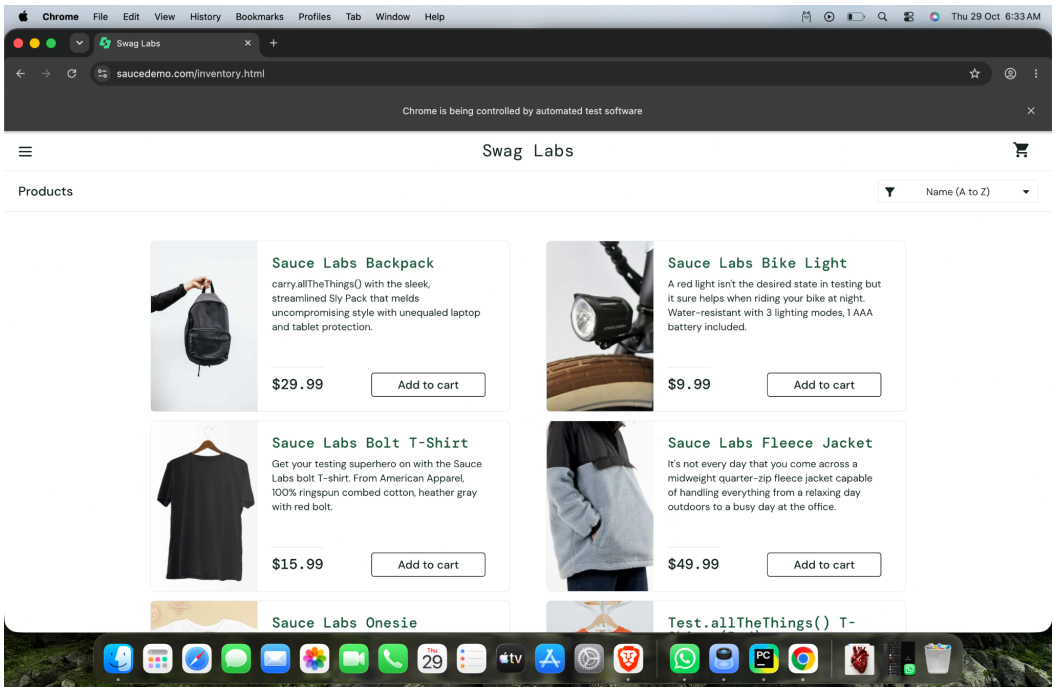
The Selenium script opens Chrome, navigates to SauceDemo, enters the supplied valid test credentials, verifies the inventory page, adds the Sauce Labs Backpack to the cart, opens the cart, proceeds to checkout, fills the checkout form, validates the checkout overview URL, and performs browser back/forward navigation. The browser is closed in a finally block so cleanup runs even if a test step fails.

Test ID	Scenario	Expected Result	Evidence / Result
TC-11-01	Open website	SauceDemo login page loads	Website flow executed
TC-11-02	Valid login	Inventory / products page opens	PASS printed in console
TC-11-03	Product selection	Backpack added to cart	PASS printed in console
TC-11-04	Cart navigation	Cart page opens	URL assertion in script
TC-11-05	Checkout form	Checkout details accepted	Form fields filled and Continue clicked
TC-11-06	Multi-page validation	Checkout overview opens	URL assertion in script
TC-11-07	Back/forward navigation	Browser history navigation completes	PASS printed by script

4. Execution Evidence

The supplied browser screenshot shows the SauceDemo inventory/products page after login. The supplied console screenshot shows a successful run of a related script labelled “AI-Assisted Login Test: PASS” and “Requirement Validation: PASS”, ending with exit code 0. These console lines are not the same output labels as the Tutorial 11 script; therefore, they are included as supporting execution evidence rather than claimed as proof that every Tutorial 11 assertion printed in the supplied code ran.

Screenshot 1 — SauceDemo Products Page



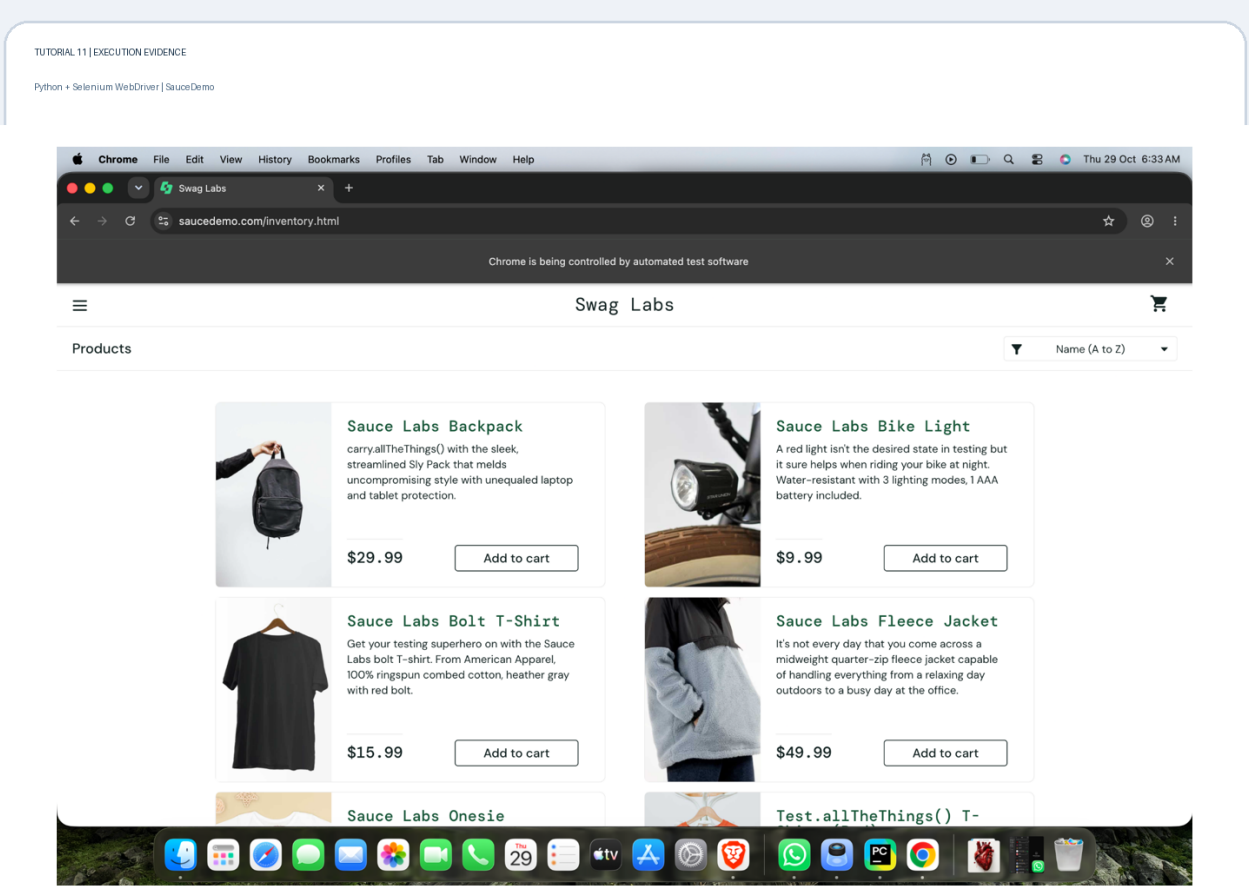
Screenshot 2 — PyCharm Console Output

```
/Users/bhaveshpatil/st_11code/.venv/bin/python /Users/bhaveshpatil/st_11code/st12.py
AI-Assisted Login Test: PASS
Requirement Validation: PASS

Process finished with exit code 0
```

5. Screenshot Collage

Combined evidence view prepared from the two screenshots supplied with the request.



Screenshot 1 ⊟ SauceDemo Products Page

```
/Users/bhaveshpatil/st_11code/.venv/bin/python /Users/bhaveshpatil/st_11code/st12.py
AI-Assisted Login Test: PASS
Requirement Validation: PASS

Process finished with exit code 0
```

Screenshot 2 ⊟ PyCharm Console / Test Output

6. Observations

• Chrome was launched under Selenium control and the SauceDemo inventory page is visible in the supplied browser screenshot.

• The provided Tutorial 11 script uses explicit waits for key page elements and URL checks for workflow validation.

• The script includes product selection, cart navigation, checkout form entry, checkout-step-two validation, and browser back/forward navigation.

• The finally block calls driver.quit() to close the browser after execution.

7. Result

The Tutorial 11 automation workflow is designed to validate login, product selection, cart access, checkout form processing, multi-page navigation, and browser history navigation on SauceDemo. The supplied screenshots provide visual evidence of the products page and a successful Python test process. For a fully auditable result, capture the console output from this exact Tutorial 11 script showing each PASS message and the final completion line.

8. Conclusion

This practical demonstrates how Selenium WebDriver can automate a browser-based shopping workflow using Python. Explicit waits and assertions help verify that the expected pages are reached, while automated form entry reduces manual effort. The exercise also introduces basic regression-oriented workflow checks through repeated navigation.

9. Professional QA Notes

• The current script validates several pages by checking URL fragments. A stronger test can also assert visible page headings and cart contents.

• The back/forward steps currently print a PASS message without checking the resulting URL after each navigation; add assertions if strict verification is required.

• Keep screenshots of the login page, product page, cart page, checkout pages, source code, and exact console output as listed in the practical manual.